1. The **application layer** is the highest abstraction layer of the TCP/IP model that provides the interfaces and protocols needed by the users. It combines the functionalities of the session layer, the presentation layer and the application layer of the OSI model. It facilitates the user to use the services of the network.
2. The **transport layer** provides a total end-to-end solution for reliable communications. TCP/IP relies on the transport layer to effectively control communications between two hosts. When an IP communication session must begin or end, the transport layer is used to build this connection.
3. The internet layer, also known as the **network layer**, controls the flow and routing of traffic to ensure data is sent speedily and accurately. This layer is also responsible for reassembling the data packet at its destination
4. The **data-link layer** identifies the network protocol type of the packet, in this instance TCP/IP. The data-link layer also provides error control and "framing." Examples of data-link layer protocols are Ethernet IEEE 802.2 framing and Point-to-Point Protocol (PPP) framing.
5. Hypertext Transfer Protocol (**HTTP**) is a method for encoding and transporting information between a client (such as a web browser) and a web server. HTTP is the primary protocol for transmission of information across the Internet.
6. A **Time Server** is a server computer that reads the actual time from a reference clock and distributes this information to its clients using a computer network. The time server may be a local network time server or an Internet time server.

ftp: used for the transfer of computer files from a server to a client on a computer network. FTP is built on a client–server model architecture using separate control and data connections between the client and the server.

Telnet is an application protocol used on the Internet or local area network to provide a bidirectional interactive text-oriented communication facility using a virtual terminal connection

Socket: network socket is a software structure within a network node of a computer network that serves as an endpoint for sending and receiving data across the network. The structure and properties of a socket are defined by an application programming interface for the networking architecture

Port: Both Socket and Port are the terms used in Transport_Layer. A port is a logical construct assigned to network processes so that they can be identified within the system.

Socket	Port
The word "Socket" is the combination of port and IP address.	The word "Port" is the number used by particular software.
It is used to identify both a machine and a service within the machine.	The same port number can be used in different computer running on same software.

What is the **DataOutputStream?**

A data output stream lets an application write primitive Java data types to an output stream in a portable way. An application can then use a data input stream to read the data back in.

What is the **DataInputStream is;**
A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream.

What is accept () in sockets
The accept () call is used by a server to accept a connection request from a client. When a connection is available, the socket created is ready for use to read data from the process that requested the connection. The call accepts the first connection on its queue of pending connections for the given socket.

byte: The byte data type is an 8-bit signed two's complement integer. It has a minimum value of -128 and a maximum value of 127 (inclusive). The byte data type can be useful for saving memory in large arrays, where the memory savings actually matters.

The **new** keyword in Java instantiates a class by allocating desired memory for an associated new object. It then returns a reference to that memory. Many times, the new keyword in Java is also used to create the array object. The new keyword is followed by a call to a constructor, which instantiates the new object.

The **read** () method of Reader Class in Java is used to read a single character from the stream. This method blocks the stream till: It has taken some input from the stream. Some IOException has occurred. It has reached the end of the stream while reading

For Example: **String s="Welcome"**; By new keyword : Java String is created by using a keyword "new". For example: String s=new String("Welcome"); It creates two objects (in String pool and in heap) and one reference variable where the variable 's' will refer to the object in the heap.

eg: String s = new String("JAVA"); This will create a new string JAVA in the main memory and also in the string pool if this string is not already present in string pool.

Java **String trim** () Method
The **trim ()** method removes whitespace from both ends of a string. Note: This method does not change the original string.

The **java.io.OutputStream.write**(byte[] b) method writes b.length bytes from the specified byte array to this output stream. The general contract for write(b) is that it should have exactly the same effect as the call write (b, 0, b.length)

getBytes() The method getBytes() encodes a String into a byte array using the platform's default charset if no argument is passed. We can pass a specific Charset to be used in the encoding process, either as a String object or a String object.

An **InputStreamReader** is a bridge from byte streams to character streams: It reads bytes and decodes them into characters using a specified charset. The charset that it uses may be specified by name or may be given explicitly, or the platform's default charset may be accepted.

getLocalPort. public int **getLocalPort**() Returns the local port number to which this socket is bound. If the socket was bound prior to being closed , then this method will continue to return the local port number after the socket is closed.

The **getPort()** function is a part of URL class. The function getPort() returns the port of a specified URL. The function returns the port number or -1 if the port is not set. Function Signature. public int getPort()

The **read()** method of Reader Class in Java is used to read a single character from the stream. This method blocks the stream till: It has taken some input from the stream. Some IOException has occurred. It has reached the end of the stream while reading

Java **BufferedReader** is a public Java class that reads text, using buffering to enable large reads at a time for efficiency, storing what is not needed immediately in memory for later use. Buffered readers are preferable for more demanding tasks, such as file and streamed readers.

The **getOutputStream()** method of Java Socket class **returns an output stream for the given socket**. If you close the returned OutputStream then it will close the linked socket.

getInputStream() method **gets the input stream of the subprocess**. The stream obtains data piped from the standard output stream of the process represented by this Process object.

The **readLine()** method of Console class in Java is **used to read a single line of text from the console**.

Unicode (UTF) − Stands for Unicode Translation Format.

writeUTF. Writes a string to the underlying output stream using modified UTF-8 encoding in a machine-independent manner